DLDCA: Performance
Week 1

September 30, 2025

Introduction

You have been introduced to the idea of pipelining which makes efficient use of the underlying hardware. In this week's lab, we'll visualize pipelining, specifically in WinMIPS64, and look at some optimizations that can be and is done is practice by compilers.

Note that this is a simulator, meaning all it does is show such an execution would look like and does not run on real hardware. It imitates how a MIPS64 processor would fetch, decode and execute instructions.

Necessary tools

Before starting the lab, ensure that wine is installed in your system. This can be verified by running the following:

```
$ wine
Usage: wine PROGRAM [ARGUMENTS...]   Run the specified program
    wine --help                   Display this help and exit
    wine --version                Output version information and exit
```

From here, ensure the executable `winmips64.exe` is present in the working directory, and run:

```
$ wine winmips64.exe
```

This should open the GUI for the simulator.

Structure of submission/templates

```
your-roll-no/
|- task1/
    |- unrolled.s (TODO)
|- task2/
    |- optimize_this.s (TODO)
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of your-roll-no.tar.gz.

The command given below can be used to do the same.

```
$ tar -cvzf your-roll-no.tar.gz your-roll-no/
```

Tasks

Task 0

We are given a file `hazards.s` to simulate on WinMIPS64.

The idea is to observe how hazards (RAW, WAW) and branches create stalls in the pipeline. By toggling settings like data forwarding, branch delay slot, branch prediction in the configuration panel, one can see how stalls are reduced.

This task mainly helps to get comfortable with the simulator UI which will be useful in further tasks.

Task 1

Let's say you are given a loop. Before executing the body of the loop, the program first checks the exit condition at the start (or the end if a check is done outside the loop too) of every iteration.

These conditions are actually branch instructions and branch instructions add stalls into the pipeline, as you would've learnt by now. Stalls can significantly reduce performance, but luckily there are a few optimisations that can be done. These can be either hardware or software, and one of the software optimisations that modern compilers employ is *Loop unrolling*.

Loop Unrolling

To start off simple, if you know that the number of iterations are even, then you can do the computations for the next iteration along with the current iteration. This looks like simply repeating the loop body once. So, two iterations of the loop are done but the exit condition is only checked once.

Reduced instructions means reduced stalls (roughly), and hence it helps with performance. This is what **unrolling** the loop generally means.

Loop unrolling is a compiler optimization that can be enabled with `-funroll-loops` in gcc. It decides which loops to unroll and by how much using some internal heuristics. In this task, you have to implement loop unrolling manually for some code that you have been given.

Note that this optimization is a trade-off between time and space, because if one hypothetically unrolls the entire loop that runs for `n` iterations, since each instruction inside the loop body will be repeated `n` times, the executable will also be roughly `n` times larger (if it's majorly comprised of loops). Bigger executables come with problems of their own, so it's important to understand the trade-off and only use it as needed.

Hence, in this task, we'll be unrolling it thrice (the body of the loop repeated thrice within the loop), 5 times, or 7 times.

You have been given a file `naive.s`. It contains a for loop which performs some computation. This loop should be unrolled as described above. The amount of unrolling to be done should be taken as input (can be 0, 1, 2, or 3). According to the input given, the corresponding amount (shown below in the table) of unrolling should be done. The modified code should be written in `unrolled.s`.

Note: The number of times the loop should run is also taken as input, and is not necessarily a multiple or 3, 5, or 7. The program should give right output at the end of it all (because after all, if program correctness is lost, what is the use of being fast!).

Loop Unrolling Options	
Input	**Unroll factor**
0	No unrolling
1	Unroll it by a factor of 3
2	Unroll it by a factor of 5
3	Unroll it by a factor of 7

Table 1: Mapping from input to unroll factor.

Task 2

You have been given an assembly file, and the task is to optimize as much as possible where the metric of concern is the IPC (Instructions Per Cycle). The amount of marks in this part would be a factor of a threshold we have and the maximum IPC achieved in the batch (the GUI itself displays CPI (Cycles per Instruction) which is just the inverse of IPC. So, lower the CPI, the better).

Note that the IPC depends not only on the number of instructions but also on how these instructions are scheduled so as to minimize hazards which directly translates to stalls.

Reordering instructions to hide load delays, reducing branches, and keeping values in registers (avoiding spills) all help improve IPC.

Thus, optimization is not just about fewer instructions, but also about pipeline-friendly code.